

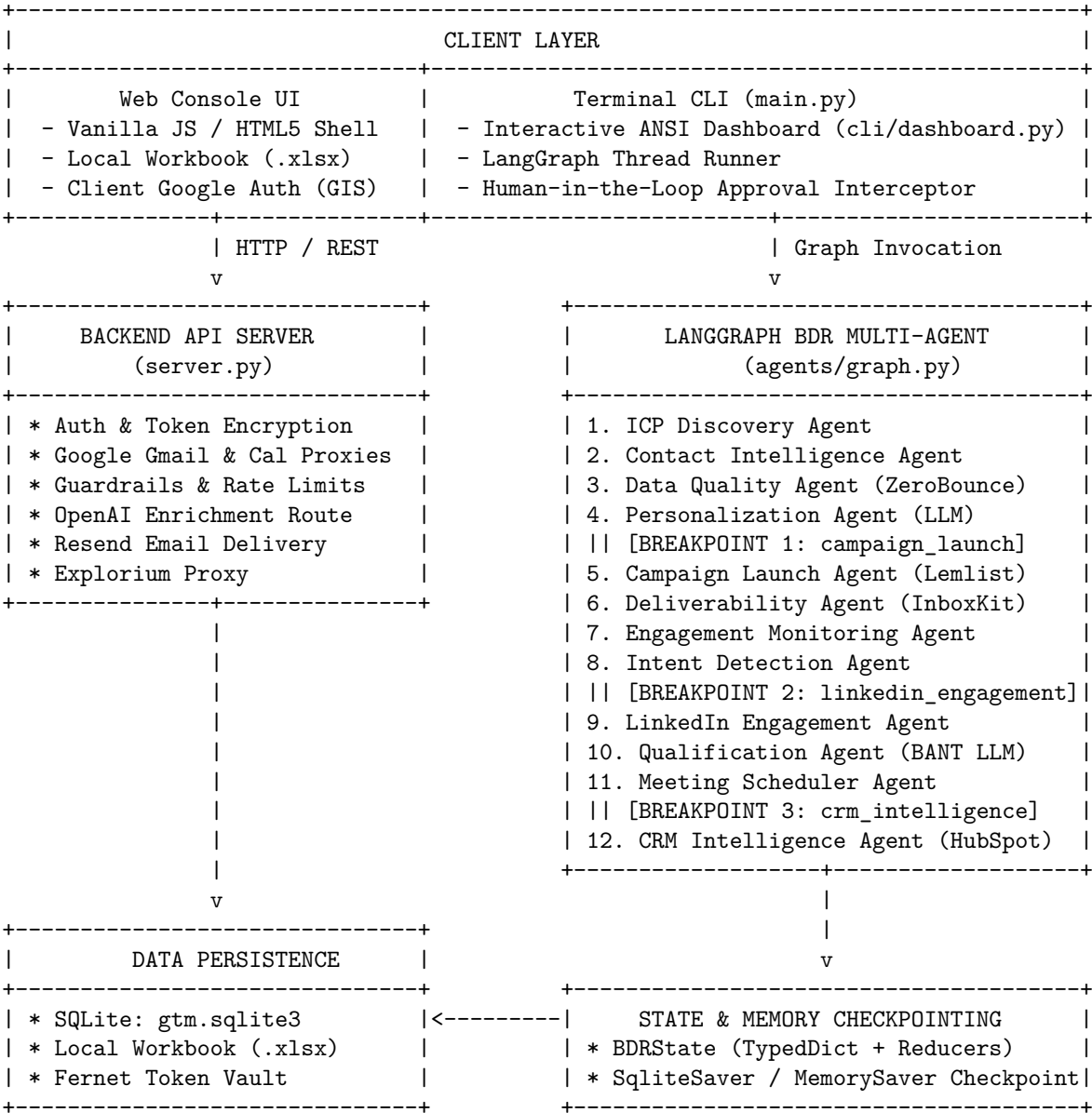
Autonomous BDR Platform & GTM Console

Comprehensive Technical Architecture & System Reference

Multi-Agent LangGraph Pipeline • REST/RPC Server • Local-First Persistence • Human-in-the-Loop Orchestration

1 System Architecture Overview

The **Autonomous BDR GTM Platform** is an enterprise-grade agentic sales development system. It pairs an asynchronous **LangGraph multi-agent pipeline** with a local-first **Python REST/RPC server** (`server.py` and Vercel serverless `api/index.py`), a terminal-based **Interactive ANSI Dashboard** (`main.py` / `cli/dashboard.py`), and a lightweight **Web Console** supporting Google Workspace OAuth2 and local workbook (`.xlsx`) persistence via the Chrome/Edge File System Access API.



2 Markdown Files & Specifications Directory

The workspace maintains strict separation of architectural concerns across dedicated markdown specifications:

File	Path	Scope, Content & Operational Guidance
README.md	./README.md	Primary manual. Details local launch (./run_local.sh, main.py), Vercel serverless functions, Google OAuth client setup, local workbook modes, Clerk gate, pilot verification, and system boundaries.
INSTALLATION.md	./INSTALLATION.md	Step-by-step developer runtime installation (Python 3.9+, Node.js, virtualenv, .env setup, and dependency management).
DESIGN.md	./DESIGN.md	Editorial design token specification. Details color palette (#292524 warm ink, #f5f5f5 canvas, pastel gradient orbs), typography (Waldenburg Light 300 serif headers, Inter body), component surfaces, and elevation rules.
AGENTS.md	./AGENTS.md	Coding agent rules governing Astryx component design: frame-first layout, avoidance of raw HTML <div>, dense data rows, and design token constraints (var(-color-*)).
elevenlabs/DESIGN.md	./elevenlabs/DESIGN.md	Specialized brand voice-AI design tokens and typography rules.
miro/DESIGN.md	./miro/DESIGN.md	Canvas visual styling, whiteboard layout models, and spatial design system tokens.

3 Tools Ecosystem & Provider Mocks

The tool architecture in tools/ supplies deterministic, testable interfaces across data enrichment, deliverability testing, cold sequencing, CRM sync, and social touches. Shared singletons are exported via tools/__init__.py.

Module	Class / Singleton	Core Methods & Operational Responsibilities
apollo_mock.py	ApolloMock	Account Discovery & Personas: search_accounts(industries, employee_range, geography) queries firmographic databases; find_contacts(domain, roles) identifies buying committees (CIO, CTO, Data Head); enrich_contact(email) supplies direct dials and seniority.
clay_mock.py	ClayMock	Signal Intelligence: get_company_signals(domain) detects funding rounds, hiring spikes, AI initiatives, and trigger events; get_contact_signals(email) parses social posts.
zerobounce_mock.py	ZeroBounceMock	Deliverability Verification: validate_email(email) classifies syntax, DNS/MX validity, and spam traps; returns status (valid, invalid, spamtrap, abuse) and health score.
inboxkit_mock.py	InboxKitMock	Sender Domain Health: get_healthy_sending_inboxes() filters active pools; get_inbox_health(email) monitors warming status and spam score; log_send(email) updates limits.
lemlist_mock.py	LemlistMock	Outreach Orchestration: add_to_sequence(contact, seq_id, subject, body) enrolls prospects; get_sequence(email) and get_all_active_sequences() provide open, click, and reply telemetry.
linkedin_mock.py	LinkedInMock	Social Touches: like_recent_post(profile_url, snippet) simulates automated engagement; send_connection_request(profile_url, note) delivers personalized notes after human review.
hubspot_mock.py	HubSpotMock	CRM Synchronization: upsert_contact(email, properties) syncs prospect attributes; update_lifecycle_stage() transitions stages; create_deal() creates deals and computes pipeline ARR.

4 Backend HTTP API & RPC Endpoints

Implemented in server.py (ProxyHTTPRequestHandler) with serverless routing via api/index.py:

Endpoint	Method	Authentication Guard	Functional Description
/api/google/oauth/start	GET	Public	Generates cryptographically random state token and redirects to Google OAuth2 consent URL with Gmail & Calendar scopes.
/api/google/oauth/callback	GET	OAuth Code & State	Verifies state expiry (< 600s), exchanges code for tokens, encrypts tokens, issues gtm_session HttpOnly cookie, and persists connection.
/api/google/status	GET	Session Cookie	Returns connection health and authenticated Google email.
/api/google/verify	GET	Session Cookie	Performs live probe on Gmail profile and Primary Calendar to assert active token permissions.
/api/google/disconnect	GET	Session Cookie	Invalidates session, drops SQLite credentials, and clears cookies.
/api/state	GET/POST	Optional / Local	Retrieves or updates serialized application database state snapshot in SQLite app_state.
/api/google/gmail/draft	POST	Session Cookie	Creates a raw RFC 2822 base64url-encoded draft in user's Gmail.
/api/google/gmail/send	POST	check_send_guardrails	Validates explicit human approval, suppression status, SHA-256 duplicate fingerprint, and daily send limit before dispatching message.
/api/google/calendar/freebusy	POST	Session Cookie	Queries Google Calendar API for free/busy intervals of attendees.
/api/google/calendar/events	POST	Session Cookie	Creates Google Calendar event with auto-generated Google Meet conference.
/api/google/gmail/replies	POST	Session Cookie	Scans Gmail for incoming messages from target prospects received within 30 days.
/api/ai/enrich	POST	Server-side OPENAI_API_KEY	Executes structured LLM analysis of prospect lists, enforcing strict anti-hallucination research rules.
/api/email/send	POST	RESEND_API_KEY	Delivers transactional outbound emails via Resend HTTPS API.
/api/proxy/*	POST	EXPLORIUM_API_KEY	Reverse proxies requests to api.explorium.ai, injecting server-held credentials safely.

4.1 Security Guardrails for Outbound Messaging

The `check_send_guardrails(payload)` routine strictly enforces multi-layer safety before executing live sends:

1. **Payload Verification:** Requires non-empty `to`, `subject`, and `body`.
2. **Suppression Enforcement:** Immediately halts if prospect has `suppressed` or `unsubscribed` flags.
3. **Mandatory Approval:** Requires strict `approved == True`.
4. **Deduplication:** Generates `SHA256(to || subject || body)` and verifies non-existence in `sent_emails`.
5. **Throttling:** Rejects sends exceeding `PROTOTYPE_DAILY_SEND_LIMIT` (default: 25/day).

5 Data Models & Persistence Architecture

5.1 SQLite Storage Model (`.prototype-data/gtm.sqlite3`)

- `app_state` (key TEXT PK, value TEXT, updated_at TEXT): Serialized state store for web console records.
- `google_connections` (id INTEGER PK, session_id, email, name, access_token, refresh_token, expires_at, updated_at): Stores OAuth tokens encrypted at rest with Fernet symmetric cipher (derived from `TOKEN_ENCRYPTION_KEY`).
- `sent_emails` (id INTEGER PK, fingerprint TEXT UNIQUE, recipient, subject, provider_id, sent_at): Audit ledger preventing duplicate message transmission.

5.2 Local Workbook Mode (`.xlsx`)

Using the browser's File System Access API, the app establishes direct two-way binding with a local Excel file containing 10 auditable sheets: `Contacts`, `Companies`, `Enrichment`, `Campaigns`, `Activities`, `Approvals`, `Events`, `Settings`, `Runs`, and `Metadata`. Provider API keys are never committed to the workbook.

6 State Schema & Memory Management

6.1 BDRState Schema Definition (`agents/state.py`)

```

1 from typing import TypedDict, List, Dict, Any, Annotated
2 import operator
3
4 class BDRState(TypedDict):
5     icp_definition: Dict[str, Any]           # Target industries, employee size, geography
6     accounts: List[Dict[str, Any]]           # Identified target accounts
7     contacts: List[Dict[str, Any]]           # Target buying committee contacts
8     sequences: Dict[str, Any]                # Active campaigns & outreach copy
9     email_outbox: List[Dict[str, Any]]        # Dispatched emails audit log
10    linkedin_actions: List[Dict[str, Any]]     # Dispatched social touches audit log
11    intent_scores: Dict[str, int]              # Calculated engagement intent scores
12    qualification_status: Dict[str, Dict]      # BANT qualification results
13    crm_records: Dict[str, Any]               # Mirrored HubSpot records (deals, meetings)
14    human_approval_queue: List[Dict[str, Any]] # Human-in-the-loop intercept queue
15    metrics: Dict[str, Any]                   # Operational telemetry metrics
16    logs: Annotated[List[str], operator.add]   # Monotonic append-only log stream
17    current_agent: str                        # Active executing agent

```

6.2 LangGraph Checkpointing & Human-in-the-Loop Interception

The workflow is compiled using `SqliteSaver` (persisted to `.prototype-data/langgraph.sqlite`) or `MemorySaver`:

```

1 compiled_graph = builder.compile(
2     checkpointer=memory,
3     interrupt_before=[
4         "campaign_launch",      # Intercept high-value outreach email copy
5         "linkedin_engagement",  # Intercept personalized LinkedIn notes
6         "crm_intelligence"      # Intercept meeting logs and deal values
7     ]
8 )

```

When execution hits an interrupted node, state is snapshotted under `thread_id`. The user inspects the queue via CLI or Web UI, mutates the state via `graph.update_state(config, ...)`, and calls `graph.invoke(None, config)` to resume.

7 LLM Configurations & Prompt Engineering

The platform standardizes on **OpenAI gpt-4o-mini** configured with `temperature=0.2` and `response_format={"type": "json_object"}` for deterministic, structured reasoning.

7.1 Core Agent Prompt Templates

1. Personalization Agent (`agents/campaign_agents.py`)

System Prompt: *You are an expert outbound BDR specializing in sales development for SaaS enterprise platforms. Generate a short, direct, highly personalized email outreach copy (1-2 paragraphs max, no generic fluff).*

User Prompt Inputs: Prospect Name, Title, Company Name, Trigger Event, AI Initiatives, Recent LinkedIn Post Snippet, Base Template.

Output Contract: `{"subject": "...", "body": "..."}"`

2. Qualification Agent (`agents/qualification_agents.py`)

System Prompt: *You are an expert sales qualifier. Evaluate the prospect's reply and details against the BANT (Budget, Authority, Need, Timeline) framework. Summarize your evaluation and output a qualification status.*

User Prompt Inputs: Prospect Full Name, Title, Company, Received Reply Text, Company Tech Stack, Trigger Signals.

Output Contract: `{"budget": str, "authority": str, "need": str, "timeline": str, "is_qualified": bool, "evaluation_summary": str}`

3. AI Research Enrichment Route (`server.py /api/ai/enrich`)

System Prompt: *You are a careful B2B research assistant.*

Instructions: *Return one profile per input contact using requested fields. Use only supplied facts and cautious inferences. Never invent phone numbers, emails, URLs, employers, events, or specific claims. Mark inferred values and leave unknown fields empty. Return JSON only.*

8 Client Invocation & RPC Execution Flow

8.1 main.py Execution Engine

1. **Initialization:** Compiles LangGraph instance and binds a unique execution session `thread_id ("bdr-<uuid>")`.
2. **First Run / Resume:** Invokes `graph.invoke(initial_state, config)` on cold launch, or `graph.invoke(None, config)` when resuming from an approval breakpoint.
3. **Interrupt Handling:** If `state_snapshot.next` hits a breakpoint node, execution yields control to `cli.dashboard.render`.
4. **Approval Processing:** Operators approve, reject, or edit content. `update_current_state()` commits approved items back into the checkpointer before the next cycle resumes.

8.2 Client-to-Server RPC Sequence

```

1 [Web Console / CLI Client]
2   |
3   | POST /api/google/gmail/send
4   | Body: { "to": "...", "subject": "...", "body": "...", "approved": true }
5   | v
6 [ProxyHTTPRequestHandler (server.py)]
7   |
8   | --- 1. Read 'gtm_session' cookie & decrypt OAuth tokens via Fernet
9   | --- 2. Auto-refresh Google access token if within 60s of expiration
10  | --- 3. Execute check_send_guardrails() (Deduplication & rate limit check)
11  | --- 4. Make HTTP RPC to Gmail API (https://gmail.googleapis.com/v1/...)
12  | --- 5. Write SHA-256 fingerprint into sent_emails table in gtm.sqlite3
13  | --- 6. Return HTTP 200 { "id": "msg_xxx", "status": "sent" }

```

9 Test Verification Suite

Test Target	Script Path	Validation Assertions
Multi-Agent Graph	tests/test_workflow.py	Asserts deterministic execution across all 12 nodes, data propagation in <code>BDRState</code> , and breakpoint halts.
Server & Guardrails	tests/test_server.py	Verifies Google OAuth flow, Fernet token cipher, SHA-256 deduplication, daily send limits, and AI proxies.
Frontend Contract	tests/test_frontend_contract.py	Asserts alignment between UI requirements, local workbook columns, and REST payloads.
Smoke Verification	scripts/verify_prototype.py	Executes full end-to-end headless run of the GTM pipeline.